문제 1) 이진 탐색 트리의 검색, 삽입, 삭제 및 분할 구현

문제 설명

이진 탐색 트리(Binary Search Tree)에 대하여 다음 기능을 완성하시오.

- BSTsearch(String K) : 키 K를 가지는 노드를 검색하여 반환, 존재하지 않으면 null을 반환.
- BSTinsert(String K) : 이진 탐색 트리의 조건에 맞는 위치에 키 K를 삽입.
- maxNode(TreeNode root) : 주어진 서브트리에서 가장 큰 키를 가지는 노드를 반환.
- delete(TreeNode root, String K) : 키 K를 가지는 노드를 삭제, 변경된 트리의 루트를 반환.
- BSTsplit(String x, BinarySearchTree SmallTree, BinarySearchTree LargeTree) : 기준 키 x를 기준으로 작은 키를 가지는 트리와 큰 키를 가지는 트리로 분할.

TreeNode, BSTdelete(), 출력 함수 및 main() 함수는 주어진 코드를 사용한다.

기본 코드

```
class TreeNode {
    String key;
    TreeNode Lchild;
    TreeNode Rchild;
}
class BinarySearchTree {
    private TreeNode rootNode;
    public TreeNode BSTsearch(String K) {
        // 코드 작성
    }
    public void BSTinsert(String K) {
        // 코드 작성
    }
    private TreeNode maxNode(TreeNode root) {
        // 코드 작성
    }
```

```
    private TreeNode delete(TreeNode root, String K) {
        // 코드 작성
    }
    public void BSTdelete(String K) {
        rootNode = delete(rootNode, K);
    }
    public boolean BSTsplit(String x, BinarySearchTree SmallTree, BinarySearchTree LargeTree) {
        // 코드 작성
    }
    private void printNode(TreeNode n) {
        if (n != null) {
            System.out.print("(");
            printNode(n.Lchild);
            System.out.print(" " + n.key + " ");
            printNode(n.Rchild);
            System.out.print(")");
        }
    }
    public void print() {
        printNode(rootNode);
        System.out.println();
    }
}
public class BSTTest {
    public static void main(String args[]) {
        BinarySearchTree t = new BinarySearchTree();
        t.BSTinsert("S");
```

```
t.BSTinsert("J");
t.BSTinsert("B");
t.BSTinsert("D");
t.BSTinsert("U");
t.BSTinsert("M");
t.BSTinsert("R");
t.BSTinsert("Q");
t.BSTinsert("A");
t.BSTinsert("G");
t.BSTinsert("E");
System.out.println("===== 초기 이진 탐색 트리 =====");
t.print();
System.out.println("===== R 검색 =====");
TreeNode n = t.BSTsearch("R");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("===== R 삭제 후 검색 =====");
t.BSTdelete("R");
n = t.BSTsearch("R");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("R 삭제 후 트리");
t.print();
```

```
System.out.println("===== E 검색 =====");
n = t.BSTsearch("E");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("===== E 삭제 후 검색 =====");
t.BSTdelete("E");
n = t.BSTsearch("E");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("E 삭제 후 트리");
t.print();
System.out.println("===== J 검색 및 삭제 =====");
n = t.BSTsearch("J");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
t.BSTdelete("J");
System.out.println("J 삭제 후 검색");
n = t.BSTsearch("J");
if (n == null)
    System.out.println("Not Found");
else
```

```
        System.out.println("Found");
System.out.println("J 삭제 후 트리");
t.print();
System.out.println("===== S 검색 및 삭제 =====");
n = t.BSTsearch("S");
if (n == null)
        System.out.println("Not Found");
else
        System.out.println("Found");
t.BSTdelete("S");
System.out.println("S 삭제 후 검색");
n = t.BSTsearch("S");
if (n == null)
        System.out.println("Not Found");
else
        System.out.println("Found");
System.out.println("S 삭제 후 트리");
t.print();
System.out.println("===== B 검색 및 삭제 =====");
n = t.BSTsearch("B");
if (n == null)
        System.out.println("Not Found");
else
        System.out.println("Found");
t.BSTdelete("B");
System.out.println("B 삭제 후 검색");
n = t.BSTsearch("B");
```

```
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("B 삭제 후 트리");
t.print();
System.out.println("===== 존재하지 않는 C 검색 =====");
n = t.BSTsearch("C");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("===== 새로운 이진 탐색 트리 t1 생성 =====");
BinarySearchTree t1 = new BinarySearchTree();
t1.BSTinsert("S");
t1.BSTinsert("J");
t1.BSTinsert("B");
t1.BSTinsert("D");
t1.BSTinsert("U");
t1.BSTinsert("M");
t1.BSTinsert("R");
t1.BSTinsert("Q");
t1.BSTinsert("A");
t1.BSTinsert("G");
t1.BSTinsert("E");
System.out.println("t1 트리");
t1.print();
```

```
System.out.println("===== t1에서 R 검색 =====");
n = t1.BSTsearch("R");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
System.out.println("===== t1에서 C 검색 =====");
n = t1.BSTsearch("C");
if (n == null)
    System.out.println("Not Found");
else
    System.out.println("Found");
BinarySearchTree Small = new BinarySearchTree();
BinarySearchTree Large = new BinarySearchTree();
System.out.println("===== t 트리를 D를 기준으로 분할 =====");
t.BSTsplit("D", Small, Large);
System.out.println("D보다 작은 트리");
Small.print();
System.out.println("D보다 큰 트리");
Large.print();
System.out.println("===== t1 트리를 C를 기준으로 분할 =====");
t1.BSTsplit("C", Small, Large);
System.out.println("C보다 작은 트리");
Small.print();
System.out.println("C보다 큰 트리");
Large.print();
System.out.println("===== T를 루트로 하는 새로운 트리 생성 =====");
```

```
t1 = new BinarySearchTree();
t1.BSTinsert("T");
t1.BSTinsert("U");
t1.BSTinsert("S");
t1.BSTinsert("R");
t1.BSTinsert("O");
t1.BSTinsert("M");
t1.BSTinsert("W");
t1.BSTinsert("X");
t1.BSTinsert("C");
t1.BSTinsert("B");
t1.BSTinsert("A");
System.out.println("T 삭제 전 트리");
t1.print();
System.out.println("===== T 삭제 =====");
t1.BSTdelete("T");
System.out.println("T 삭제 후 트리");
t1.print();
System.out.println("===== X, R 삭제 테스트를 위한 새로운 트리 생성 =====");
t1 = new BinarySearchTree();
t1.BSTinsert("T");
t1.BSTinsert("U");
t1.BSTinsert("R");
t1.BSTinsert("S");
t1.BSTinsert("O");
t1.BSTinsert("M");
t1.BSTinsert("X");
```

```
        t1.BSTinsert("W");
        t1.BSTinsert("C");
        t1.BSTinsert("B");
        t1.BSTinsert("Y");
        System.out.println("삭제 전 트리");
        t1.print();
        System.out.println("===== X 삭제 =====");
        t1.BSTdelete("X");
        System.out.println("X 삭제 후 트리");
        t1.print();
        System.out.println("===== R 삭제 =====");
        t1.BSTdelete("R");
        System.out.println("R 삭제 후 트리");
        t1.print();
    }
}
```

<실행결과 & main( )에서 만들어지는 트리>

```
===== 초기 이진 탐색 트리 =====
(((( A ) B ( D (( E ) G ))) J ( M (( Q ) R ))) S ( U ))
===== R 검색 =====
Found
===== R 삭제 후 검색 =====
Not Found
R 삭제 후 트리
(((( A ) B ( D (( E ) G ))) J ( M ( Q ))) S ( U ))
===== E 검색 =====
Found
===== E 삭제 후 검색 =====
Not Found
E 삭제 후 트리
(((( A ) B ( D ( G ))) J ( M ( Q ))) S ( U ))
===== J 검색 및 삭제 =====
Found
J 삭제 후 검색
Not Found
J 삭제 후 트리
(((( A ) B ( D )) G ( M ( Q ))) S ( U ))
===== S 검색 및 삭제 =====
Found
S 삭제 후 검색
Not Found
S 삭제 후 트리
(((( A ) B ( D )) G ( M )) Q ( U ))
===== B 검색 및 삭제 =====
Found
B 삭제 후 검색
Not Found
B 삭제 후 트리
((( A ( D )) G ( M )) Q ( U ))
===== 존재하지 않는 C 검색 =====
Not Found
```

```
===== 새로운 이진 탐색 트리 t1 생성 =====
t1 트리
(((( A ) B ( D (( E ) G ))) J ( M (( Q ) R ))) S ( U ))
===== t1에서 R 검색 =====
Found
===== t1에서 C 검색 =====
Not Found
===== t 트리를 D를 기준으로 분할 =====
D보다 작은 트리
( A )
D보다 큰 트리
(( G ( M )) Q ( U ))
===== t1 트리를 C를 기준으로 분할 =====
C보다 작은 트리
(( A ) B )
C보다 큰 트리
((( D (( E ) G )) J ( M (( Q ) R ))) S ( U ))
===== T를 루트로 하는 새로운 트리 생성 =====
T 삭제 전 트리
(((((((( A ) B ) C ) M ) O ) R ) S ) T ( U ( W ( X ))))
===== T 삭제 =====
T 삭제 후 트리
(((((((( A ) B ) C ) M ) O ) R ) S ( U ( W ( X ))))
===== X, R 삭제 테스트를 위한 새로운 트리 생성 =====
삭제 전 트리
((((((( B ) C ) M ) O ) R ( S )) T ( U (( W ) X ( Y ))))
===== X 삭제 =====
X 삭제 후 트리
((((((( B ) C ) M ) O ) R ( S )) T ( U ( W ( Y ))))
===== R 삭제 =====
R 삭제 후 트리
(((((( B ) C ) M ) O ( S )) T ( U ( W ( Y ))))
```